Họ và tên	Lê Chuẩn
MSSV	23695731
Lớp	DHKTMP19B
Link Repo	https://github.com/Fangyuangreatlove/TBDD_Exercise.git

BÀI 01 – Promise trả về "Hello Async" sau 2 giây

Yêu cầu đề bài: Create a Promise that returns the string "Hello Async" after 2 seconds.

SOURCE CODE

```
// BÀI 01: Tạo Promise trả về chuỗi "Hello Async" sau 2 giây
export function bai01(): Promise<string> {
  return new Promise<string>((resolve) => {
    setTimeout(() => {
      resolve("Hello Async");
    }, 2000);
  });
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Bài toán minh họa cách khởi tạo một đối tượng Promise cơ bản trong TypeScript/JavaScript nhằm quản lý các tác vụ bất đồng bộ (ở đây là bộ định thời gian setTimeout).

2. Phân tích chi tiết mã nguồn:

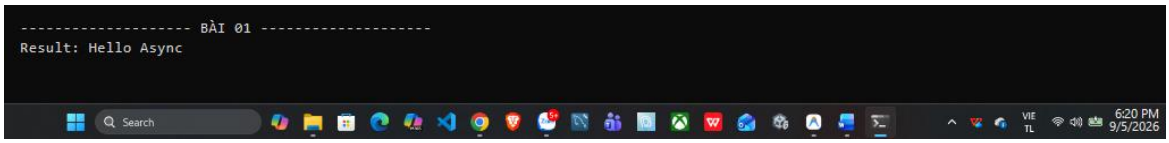
- new Promise<string>((resolve) => {...}): Hàm khởi tạo nhận vào một callback executor với tham số resolve. Kiểu trả về được xác định rõ là Promise<string>.

- setTimeout(..., 2000): Thiết lập bộ đếm thời gian 2 giây (2000ms). Trong thời gian này, Promise nằm ở trạng thái 'pending'.

- Khi hết 2000ms, bộ đếm thời gian hoàn tất và hàm resolve("Hello Async") được đẩy vào Microtask Queue để kích hoạt, chuyển trạng thái Promise sang 'fulfilled' với giá trị kết quả là chuỗi "Hello Async".

3. Ứng dụng thực tế: Trong lập trình di động, cơ chế này giúp trì hoãn tác vụ (như hiển thị màn hình Splash Screen, delay hiệu ứng) mà không làm đơ giao diện người dùng (không chặn Main UI Thread).

ẢNH CHỤP KẾT QUẢ



BÀI 02 – Promise resolve số 10 sau 1 giây

Yêu cầu đề bài: Write a function that returns a Promise resolving with the number 10 after 1 second.

SOURCE CODE

```
// BÀI 02: Viết hàm trả về Promise resolving với số 10 sau 1 giây
export function bai02(): Promise<number> {
  return new Promise<number>((resolve) => {
    setTimeout(() => {
      resolve(10);
    }, 1000);
  });
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Xây dựng một hàm trả về một Promise có kiểu dữ liệu số (number) giải quyết (resolve) sau 1 giây.

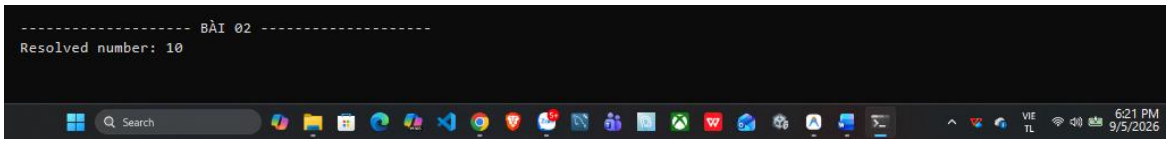
2. Phân tích chi tiết mã nguồn:

- Hàm bai02() được định nghĩa với kiểu trả về tường minh là Promise<number>, đảm bảo TypeScript kiểm soát chặt chẽ kiểu dữ liệu của giá trị resolve.

- Bên trong hàm, setTimeout gọi resolve(10) sau đúng 1000 mili-giây. Giá trị 10 được chuyển đến các bên tiêu thụ (thông qua .then() hoặc await).

3. Cơ chế hoạt động: Khi gọi bai02(), một đối tượng Promise lập tức được trả về ở trạng thái pending. JavaScript Event Loop tiếp tục xử lý các luồng khác. Đúng 1s sau, timer hết hạn và callback giải quyết Promise với giá trị 10.

ẢNH CHỤP KẾT QUẢ



BÀI 03 – Promise reject lỗi "Something went wrong" sau 1 giây

Yêu cầu đề bài: Write a function that rejects a Promise with the error "Something went wrong" after 1 second.

SOURCE CODE

```
// BÀI 03: Viết hàm rejects một Promise với lỗi "Something went wrong" sau 1 giây
export function bai03(): Promise<never> {
  return new Promise<never>((_, reject) => {
    setTimeout(() => {
      reject(new Error("Something went wrong"));
    }, 1000);
  });
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Minh họa cơ chế xử lý trường hợp thất bại (Rejection) của một Promise bằng hàm reject và lớp Error.

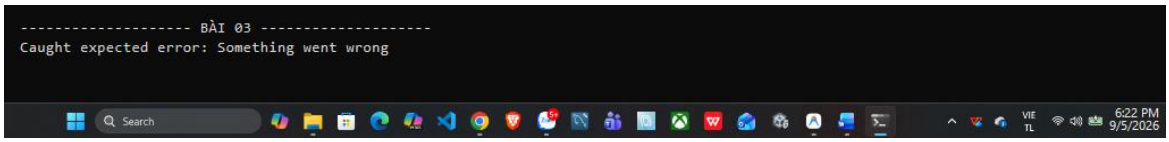
2. Phân tích chi tiết mã nguồn:

- Tham số thứ hai của hàm executor trong Promise là reject. Kiểu trả về được gán Promise<never> vì Promise này luôn bị từ chối và không trả về giá trị thành công.

- Sau 1000ms, hàm gọi reject(new Error("Something went wrong")). Thao tác này đưa Promise từ trạng thái pending sang 'rejected' đi kèm một instance lỗi chuẩn Error chứa message giải thích.

3. Ý nghĩa thực tế: Giúp mô phỏng các lỗi thường gặp trong ứng dụng di động như mất kết nối mạng, server trả về mã lỗi 500 hoặc truy vấn thất bại, từ đó xây dựng các khối xử lý ngoại lệ an toàn.

ẢNH CHỤP KẾT QUẢ



BÀI 04 – Dùng .then() và .catch() xử lý Promise sinh số ngẫu nhiên

Yêu cầu đề bài: Use .then() and .catch() to handle a Promise that returns a random number.

SOURCE CODE

```
// BÀI 04: Dùng .then() và .catch() xử lý Promise trả về số ngẫu nhiên
export function getRandomNumber(): Promise<number> {
  return new Promise<number>((resolve, reject) => {
    const num = Math.random();
    if (num >= 0.3) {
      resolve(num);
    } else {
      reject(new Error(`Number too small: ${num.toFixed(4)} (< 0.3)`));
    }
  });
}

export function bai04(): Promise<void> {
  return getRandomNumber()
    .then((result) => {
      console.log("[Success] Random number:", result.toFixed(4));
    })
    .catch((error: Error) => {
      console.log("[Handled Error]:", error.message);
    });
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Rèn luyện kỹ năng tiêu thụ Promise bằng cặp phương thức chuẩn .then() và .catch().

2. Phân tích chi tiết mã nguồn:

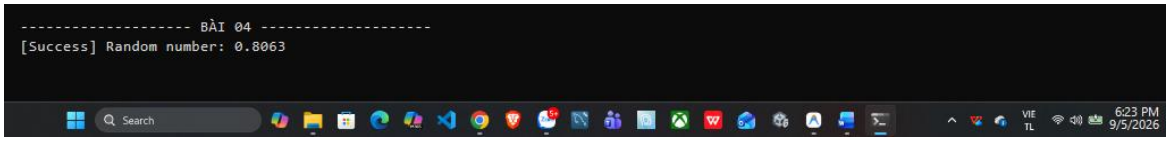
- Hàm getRandomNumber() sinh số ngẫu nhiên Math.random() trong đoạn [0, 1). Nếu giá trị >= 0.3, hàm gọi resolve(num) để báo thành công. Ngược lại (< 0.3), hàm gọi reject(new Error(...)) để báo lỗi số quá nhỏ.

- Hàm tiêu thụ bai04() gắn .then((result) => {...}) để tiếp nhận giá trị thành công và in ra console.

- Phương thức .catch((error) => {...}) được nối tiếp ngay phía sau để tóm gọn mọi biệt lệ nếu Promise bị reject.

3. Ý nghĩa thực tế: Đây là mô hình kinh điển để giao tiếp với các API không chắc chắn, bảo đảm ứng dụng di động không bị crash đột ngột khi xảy ra lỗi ngoài ý muốn.

ẢNH CHỤP KẾT QUẢ



BÀI 05 – Hàm simulateTask(time) trả về "Task done" sau time ms

Yêu cầu đề bài: Create a function simulateTask(time) that returns a Promise resolving with "Task done" after time ms.

SOURCE CODE

```
// BÀI 05: Hàm simulateTask(time) trả về Promise resolving với "Task done" sau time ms
export function simulateTask(time: number): Promise<string> {
  return new Promise<string>((resolve) => {
    setTimeout(() => {
      resolve("Task done");
    }, time);
  });
}

export const bai05 = simulateTask;
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Xây dựng hàm mô phỏng tiến trình simulateTask(time) có tính tái sử dụng cao cho toàn bộ đồ án.

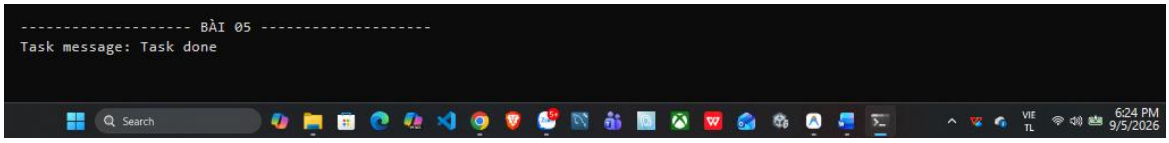
2. Phân tích chi tiết mã nguồn:

- Hàm nhận đối số đầu vào time kiểu number (đơn vị mili-giây).

- Sau khoảng thời gian time truyền vào, Promise resolve chuỗi "Task done".

3. Ý nghĩa thực tế: Hàm này tương đương với một tác vụ I/O giả lập (đọc ghi tệp tin, truy xuất cơ sở dữ liệu SQLite/Realm trên điện thoại, tải ảnh từ server) với thời gian phản hồi tùy biến theo từng bài toán kiểm thử.

ẢNH CHỤP KẾT QUẢ



BÀI 06 – Dùng Promise.all() chạy 3 Promise song song

Yêu cầu đề bài: Use Promise.all() to run 3 simulated Promises in parallel and print the result.

SOURCE CODE

```
// BÀI 06: Dùng Promise.all() chạy 3 Promise song song
import { simulateTask } from "./bai05";

export function bai06(): Promise<void> {
  const p1 = simulateTask(1000).then((res) => `Task 1 (1000ms): ${res}`);
  const p2 = simulateTask(1500).then((res) => `Task 2 (1500ms): ${res}`);
  const p3 = simulateTask(500).then((res) => `Task 3 (500ms): ${res}`);

  return Promise.all([p1, p2, p3]).then((results) => {
    console.log("Promise.all results:", results);
  });
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Sử dụng Promise.all() để thực thi song song nhiều tác vụ bất đồng bộ độc lập nhằm tối ưu hóa thời gian chạy.

2. Phân tích chi tiết mã nguồn:

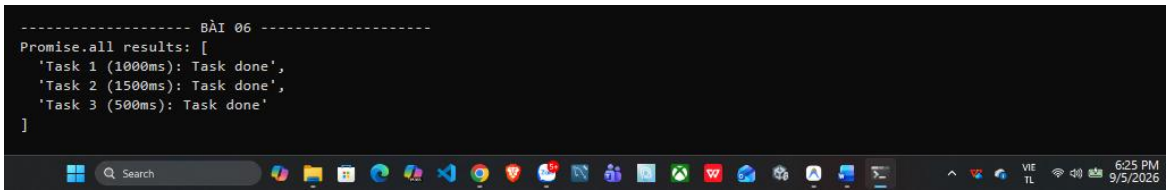
- Ba tác vụ p1 (1000ms), p2 (1500ms), p3 (500ms) được kích hoạt cùng lúc và gom vào mảng truyền cho Promise.all([p1, p2, p3]).

- Cả 3 tác vụ cùng chạy song song trong nền. Tổng thời gian hoàn thành của Promise.all chỉ bằng thời gian của tác vụ dài nhất là 1500ms (thay vì 1000 + 1500 + 500 = 3000ms nếu chạy tuần tự).

- Mảng kết quả results trả về đúng theo thứ tự khai báo [Task 1, Task 2, Task 3].

3. Ứng dụng thực tế: Thường dùng khi khởi động ứng dụng di động cần tải đồng thời thông tin người dùng, danh mục sản phẩm và cấu hình hệ thống.

ẢNH CHỤP KẾT QUẢ



BÀI 07 – Dùng Promise.race() lấy kết quả Promise về đích trước

Yêu cầu đề bài: Use Promise.race() to return whichever Promise resolves first.

SOURCE CODE

```
// BÀI 07: Dùng Promise.race() trả về Promise nào resolve trước
export function bai07(): Promise<void> {
  const p1 = new Promise<string>((resolve) =>
    setTimeout(() => resolve("Task 1 completed (1500ms)"), 1500)
  );
  const p2 = new Promise<string>((resolve) =>
    setTimeout(() => resolve("Task 2 completed (500ms - WINNER)"), 500)
  );
  const p3 = new Promise<string>((resolve) =>
    setTimeout(() => resolve("Task 3 completed (2000ms)"), 2000)
  );

  return Promise.race([p1, p2, p3]).then((winner) => {
    console.log("Fastest result (Promise.race):", winner);
  });
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Áp dụng phương thức Promise.race() để lấy kết quả của tác vụ phản hồi nhanh nhất.

2. Phân tích chi tiết mã nguồn:

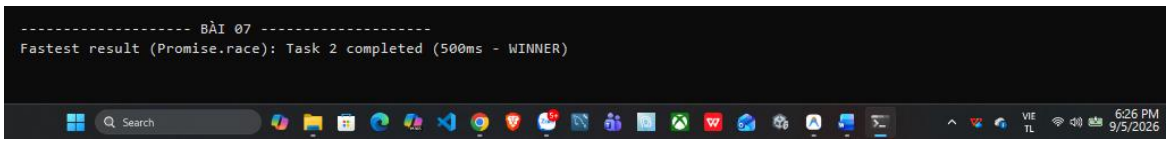
- Khởi tạo 3 Promise có thời gian trễ khác nhau: p1 (1500ms), p2 (500ms) và p3 (2000ms).

- Promise.race([p1, p2, p3]) giám sát cả 3 tác vụ. Ngay khi có tác vụ đầu tiên chuyển trạng thái (settle), Promise.race lập tức chuyển trạng thái theo tác vụ đó và bỏ qua các kết quả đến sau.

- Vì p2 chỉ mất 500ms nên chuỗi thông báo của Task 2 được in ra làm kết quả chiến thắng.

3. Ứng dụng thực tế: Áp dụng khi gửi request đồng thời tới nhiều máy chủ dự phòng (CDN/Mirror) để lấy kết quả từ server phản hồi nhanh nhất, tối ưu trải nghiệm người dùng di động.

ẢNH CHỤP KẾT QUẢ



BÀI 08 – Chuỗi Promise chain: bình phương 2, nhân đôi, cộng 5

Yêu cầu đề bài: Create a Promise chain: square the number 2, then double it, then add 5.

SOURCE CODE

```
// BÀI 08: Tạo chuỗi Promise: bình phương số 2, nhân đôi, cộng 5
export function bai08(): Promise<void> {
  return Promise.resolve(2)
    .then((n) => {
      const square = n * n; // 2 * 2 = 4
      console.log(`Step 1 - Square of ${n}: ${square}`);
      return square;
    })
    .then((n) => {
      const doubled = n * 2; // 4 * 2 = 8
      console.log(`Step 2 - Double: ${doubled}`);
      return doubled;
    })
    .then((n) => {
      const finalResult = n + 5; // 8 + 5 = 13
      console.log(`Step 3 - Add 5: ${finalResult}`);
      return finalResult;
    })
    .then((result) => {
      console.log(`Final Chain Result: ${result}`);
    });
}
```

GIẢI THÍCH

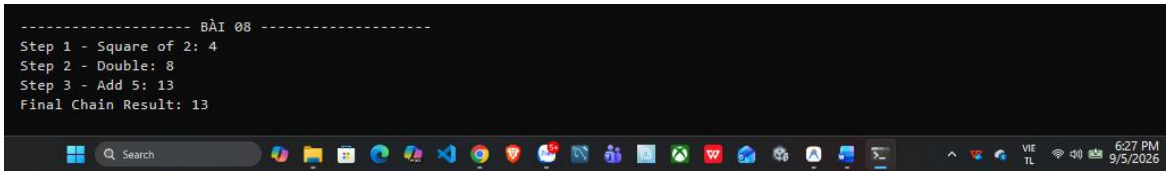
1. Khái niệm và mục tiêu: Xây dựng chuỗi Promise Chain để biến đổi dữ liệu qua nhiều giai đoạn tính toán liên tiếp.

2. Phân tích chi tiết mã nguồn:

- Khởi tạo giá trị bắt đầu bằng Promise.resolve(2).
- Bước 1 (.then thứ nhất): Lấy số 2 bình phương thành 4 và return 4.
- Bước 2 (.then thứ hai): Nhận giá trị 4 từ bước 1, nhân đôi thành 8 và return 8.
- Bước 3 (.then thứ ba): Nhận giá trị 8 từ bước 2, cộng thêm 5 thành 13 và return 13.
- Bước 4: Nhận kết quả 13 cuối cùng và in ra màn hình.

3. Ý nghĩa thực tế: Thể hiện nguyên lý xử lý dữ liệu qua đường ống (Data Pipeline), mỗi bước độc lập và chuyển tiếp dữ liệu an toàn mà không rơi vào tình trạng lồng callback (Callback Hell).

ẢNH CHỤP KẾT QUẢ



BÀI 09 – Đọc mảng sau 1 giây và lọc số chẵn

Yêu cầu đề bài: Write a Promise that reads an array after 1 second and filters even numbers.

SOURCE CODE

```
// BÀI 09: Promise đọc mảng sau 1 giây và lọc số chẵn
export function bai09(): Promise<number[]> {
  const numbers: number[] = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

  return new Promise<number[]>((resolve) => {
    setTimeout(() => {
      const evens = numbers.filter((num) => num % 2 === 0);
      resolve(evens);
    }, 1000);
  }).then((filtered) => {
    console.log("Filtered even numbers:", filtered);
    return filtered;
  });
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Mô phỏng việc đọc dữ liệu danh sách mảng số nguyên bất đồng bộ và thực hiện lọc dữ liệu bằng hàm bậc cao filter.

2. Phân tích chi tiết mã nguồn:

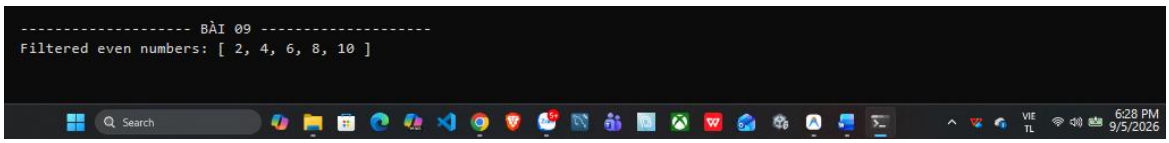
- Giả lập việc truy vấn mảng số nguyên [1..10] tốn 1000ms qua setTimeout.

- Sau 1 giây, callback tiến hành lọc dữ liệu bằng biểu thức predicate (num % 2 === 0), giữ lại các số chia hết cho 2 là [2, 4, 6, 8, 10].

- Dữ liệu đã lọc được đưa vào resolve() và chuyển tiếp tới .then() để in ra kết quả.

3. Ứng dụng thực tế: Đây là tác vụ rất phổ biến khi ứng dụng đọc danh sách từ cơ sở dữ liệu SQLite/Realm trên thiết bị di động rồi áp dụng các bộ lọc tìm kiếm theo yêu cầu của người dùng.

ẢNH CHỤP KẾT QUẢ



BÀI 10 – Dùng .finally() log "Done" khi Promise kết thúc

Yêu cầu đề bài: Use .finally() to log "Done" when a Promise finishes (success or failure).

SOURCE CODE

```
// BÀI 10: Dùng .finally() log "Done" khi Promise hoàn tất
import { simulateTask } from "./bai05";

export function bai10(): Promise<void> {
  return simulateTask(1000)
    .then((msg) => {
      console.log("Task resolved with:", msg);
    })
    .catch((err: Error) => {
      console.log("Task rejected with:", err.message);
    })
    .finally(() => {
      console.log("Done");
    });
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Sử dụng phương thức .finally() để thực thi các thao tác dọn dẹp hoặc kết thúc vòng đời của Promise.

2. Phân tích chi tiết mã nguồn:

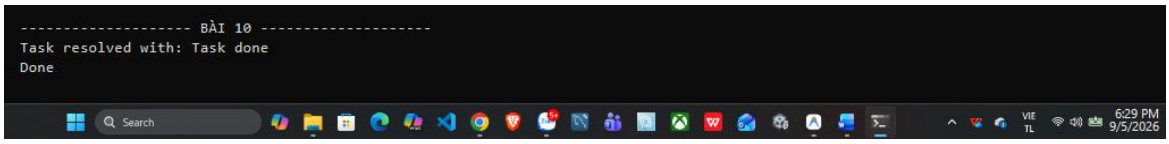
- Promise được gắn lần lượt các phương thức .then() (đón nhận kết quả), .catch() (bắt lỗi nếu có) và cuối cùng là .finally().

- Đặc điểm quan trọng của .finally() là luôn luôn được kích hoạt khi Promise bước vào trạng thái settled (dù thành công hay thất bại).

- Callback trong .finally() in ra thông báo 'Done'.

3. Ứng dụng thực tế: Trong ứng dụng di động, .finally() luôn được dùng để ẩn biểu tượng quay tròn đang tải (ActivityIndicator / Loading Spinner) hoặc đóng kết nối socket.

ẢNH CHỤP KẾT QUẢ



BÀI 11 – Chuyển đổi Bài 1 sang cú pháp async/await

Yêu cầu đề bài: Convert Exercise 1 into async/await.

SOURCE CODE

```
// BÀI 11: Chuyển đổi Bài 1 sang cú pháp async/await
import { bai01 } from "./bai01";

export async function bai11(): Promise<string> {
  const result = await bai01();
  console.log("Result using async/await:", result);
  return result;
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Chuyển đổi mã nguồn sử dụng Promise truyền thống từ Bài 1 sang cú pháp async/await hiện đại của ES2017+.

2. Phân tích chi tiết mã nguồn:

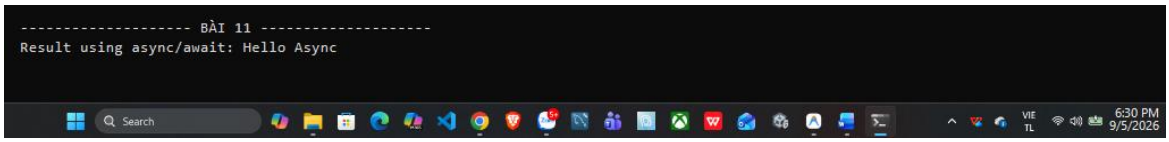
- Hàm bai11 được khai báo với từ khóa async ở đầu, tự động trả về một Promise<string>.

- Từ khóa await được đặt trước lời gọi bai01(). Lệnh này tạm dừng tiến trình của hàm cho đến khi Promise giải quyết xong và trích xuất trực tiếp giá trị chuỗi "Hello Async".

- Giá trị được gán thẳng vào biến message và log ra console.

3. Ưu điểm vượt trội: Cú pháp async/await biến code bất đồng bộ trông giống hệt code đồng bộ tuần tự, dễ debug, dễ đặt breakpoint và loại bỏ hoàn toàn việc lồng các hàm callback .then().

ẢNH CHỤP KẾT QUẢ



BÀI 12 – Async function gọi simulateTask(2000) và log kết quả

Yêu cầu đề bài: Write an async function that calls simulateTask(2000) and logs the result.

SOURCE CODE

```
// BÀI 12: Async function gọi simulateTask(2000) và log kết quả
import { simulateTask } from "./bai05";

export async function bai12(): Promise<void> {
  console.log("Calling simulateTask(2000)...");
  const result = await simulateTask(2000);
  console.log("Received result:", result);
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Viết một hàm async gọi hàm simulateTask với thời gian trễ 2000ms và hiển thị kết quả.

2. Phân tích chi tiết mã nguồn:

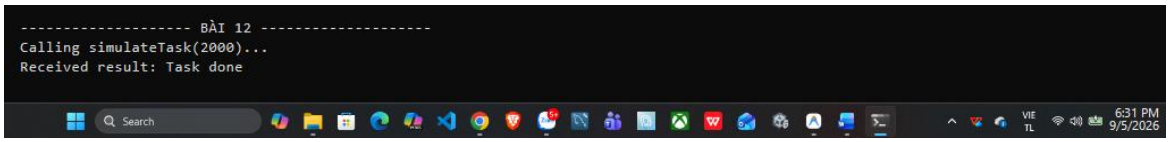
- Hàm bai12() được khai báo kiểu async trả về Promise<void>.

- await simulateTask(2000) chặn dòng thực thi bên trong hàm bai12 trong vòng 2 giây cho đến khi Promise trả về kết quả "Task done".

- Sau đó, lệnh in kết quả mới được kích hoạt.

3. Ý nghĩa thực tế: Giúp làm quen với việc phối hợp linh hoạt giữa các hàm trả về Promise truyền thống với cú pháp async/await trong các module dự án lớn.

ẢNH CHỤP KẾT QUẢ



BÀI 13 – Bắt lỗi bằng try/catch với async/await

Yêu cầu đề bài: Handle errors using try/catch with async/await.

SOURCE CODE

```
// BÀI 13: Xử lý lỗi bằng khối try/catch kết hợp async/await
import { bai03 } from "./bai03";

export async function bai13(): Promise<void> {
  try {
    console.log("Attempting risky async operation...");
    await bai03();
  } catch (error: any) {
    console.log("Successfully handled error in catch block:", error.message);
  }
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Cơ chế xử lý lỗi đồng nhất bằng khối try/catch khi lập trình với async/await.

2. Phân tích chi tiết mã nguồn:

- Khi một tác vụ bất đồng bộ được gọi với await (cụ thể là hàm bai03() cố tình reject lỗi),

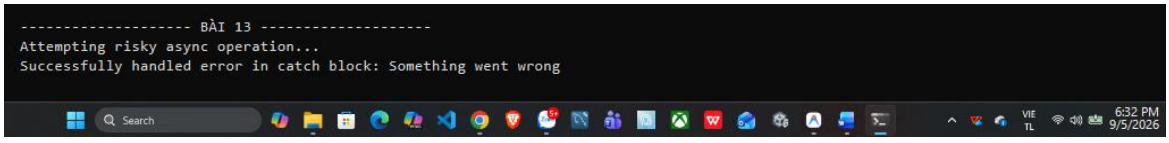
nếu Promise bị reject, JavaScript Engine sẽ tự động ném ra (throw) một ngoại lệ tại vị trí await đó.

- Khối try bao bọc lời gọi này, lập tức chuyển luồng điều khiển sang khối catch(error).

- Trong khối catch, ta truy xuất thuộc tính error.message và hiển thị thông báo lỗi đã được kiểm soát an toàn.

3. Ý nghĩa thực tế: Giúp gom toàn bộ các lỗi bất đồng bộ vào cấu trúc try/catch quen thuộc, tránh tình trạng UnhandledPromiseRejection làm sập server hoặc ứng dụng.

ẢNH CHỤP KẾT QUẢ



BÀI 14 – Async function nhận vào số, đợi 1s và trả về số × 3

Yêu cầu đề bài: Write an async function that takes a number, waits 1 second, and returns the number × 3.

SOURCE CODE

```
// BÀI 14: Async function nhận số, đợi 1 giây, trả về số x 3
export async function multiplyByThree(num: number): Promise<number> {
  await new Promise((resolve) => setTimeout(resolve, 1000));
  return num * 3;
}

export async function bai14(): Promise<void> {
  const input = 5;
  const output = await multiplyByThree(input);
  console.log(`Input: ${input} -> After 1s (x3): ${output}`);
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Viết một async function nhận vào tham số số học, tạo độ trễ 1 giây và trả về kết quả nhân 3.

2. Phân tích chi tiết mã nguồn:

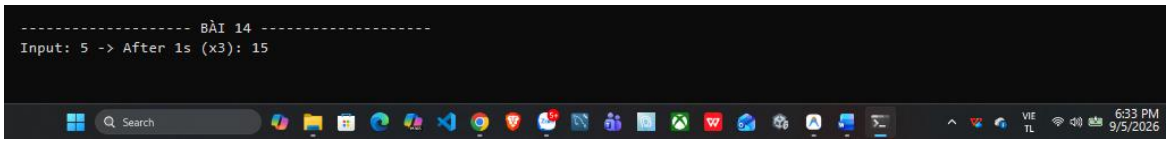
- Hàm multiplyByThree(num: number): Promise<number> nhận một đối số số học.

- Lệnh await new Promise(resolve => setTimeout(resolve, 1000)) đóng vai trò như một lệnh sleep(1000) tạm dừng luồng 1 giây.

- Sau 1 giây, biểu thức num * 3 được tính toán và return. Giá trị này tự động được TypeScript gói vào Promise<number>.

3. Ứng dụng thực tế: Mô phỏng các tác vụ tính toán nặng cần thời gian xử lý hoặc các hàm biến đổi dữ liệu trả về từ server trước khi render lên UI.

ẢNH CHỤP KẾT QUẢ



BÀI 15 – Gọi nhiều async function tuần tự bằng await

Yêu cầu đề bài: Call multiple async functions sequentially using await.

SOURCE CODE

```
// BÀI 15: Gọi nhiều async function tuần tự bằng await
import { multiplyByThree } from "./bai14";

export async function bai15(): Promise<void> {
  console.log("Starting sequential async calls...");
  const t0 = Date.now();

  const step1 = await multiplyByThree(2); // 2 * 3 = 6 (1s)
  console.log(`Step 1: ${step1} (after ${Date.now() - t0}ms)`);

  const step2 = await multiplyByThree(step1); // 6 * 3 = 18 (1s)
  console.log(`Step 2: ${step2} (after ${Date.now() - t0}ms)`);

  const step3 = await multiplyByThree(step2); // 18 * 3 = 54 (1s)
  console.log(`Step 3: ${step3} (after ${Date.now() - t0}ms)`);
}
```

GIẢI THÍCH

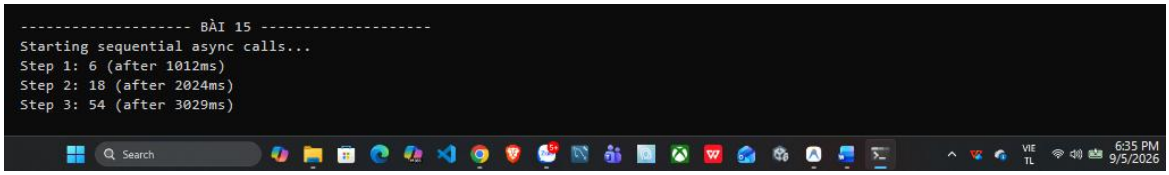
1. Khái niệm và mục tiêu: Minh họa việc gọi tuần tự (sequential execution) nhiều async function bằng await liên tiếp.

2. Phân tích chi tiết mã nguồn:

- Bước 1: await multiplyByThree(2) mất 1 giây -> kết quả 6.
- Bước 2: await multiplyByThree(6) mất thêm 1 giây -> kết quả 18.
- Bước 3: await multiplyByThree(18) mất thêm 1 giây -> kết quả 54.
- Tổng thời gian chạy đo được là hơn 3000ms (~3 giây).

3. Phân tích hiệu năng: Cách gọi tuần tự là bắt buộc khi tác vụ sau phụ thuộc vào kết quả của tác vụ trước (ví dụ: đăng nhập lấy token -> dùng token lấy user profile -> dùng user profile lấy danh sách bài viết).

ẢNH CHỤP KẾT QUẢ



BÀI 16 – Gọi nhiều async function song song bằng Promise.all()

Yêu cầu đề bài: Call multiple async functions in parallel using Promise.all().

SOURCE CODE

```
// BÀI 16: Gọi nhiều async function song song bằng Promise.all()
import { multiplyByThree } from "./bai14";

export async function bai16(): Promise<void> {
  console.log("Starting parallel async calls...");
  const t0 = Date.now();

  const results = await Promise.all([
    multiplyByThree(1),
    multiplyByThree(2),
    multiplyByThree(3)
  ]);

  console.log("Parallel results:", results);
  console.log(`Total execution time: ${Date.now() - t0}ms (~1 second)`);
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Kết hợp cú pháp async/await với Promise.all() để thực thi song song nhiều tác vụ độc lập.

2. Phân tích chi tiết mã nguồn:

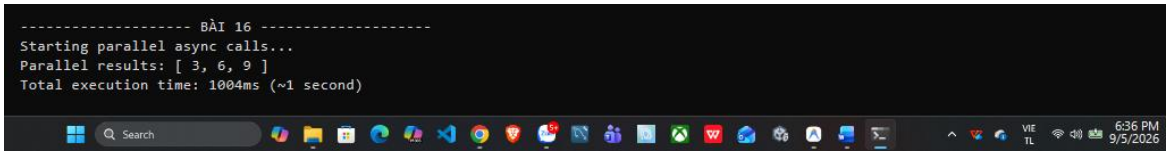
- Mảng gồm 3 lời gọi hàm multiplyByThree(1), multiplyByThree(2), multiplyByThree(3) được truyền vào Promise.all.

- Cả 3 hàm đều được kích hoạt cùng một lúc. Nhờ từ khóa await trước Promise.all, hàm chờ cho đến khi cả 3 tác vụ đều hoàn tất.

- Kết quả trả về là mảng [3, 6, 9]. Tổng thời gian thực thi chỉ mất khoảng 1004ms (~1 giây), nhanh gấp 3 lần so với cách gọi tuần tự ở Bài 15.

3. Bài học kinh nghiệm: Luôn ưu tiên dùng Promise.all cho các tác vụ độc lập để giảm thời gian chờ đợi của người dùng ứng dụng di động.

ẢNH CHỤP KẾT QUẢ



BÀI 17 – Dùng for await...of duyệt qua mảng Promise

Yêu cầu đề bài: Use for await...of to iterate over an array of Promises.

SOURCE CODE

```
// BÀI 17: Sử dụng for await...of để duyệt qua mảng Promises
export async function bai17(): Promise<void> {
  const promises: Promise<string>[] = [
    new Promise((resolve) => setTimeout(() => resolve("Item 1 (400ms)"), 400)),
    new Promise((resolve) => setTimeout(() => resolve("Item 2 (200ms)"), 200)),
    new Promise((resolve) => setTimeout(() => resolve("Item 3 (300ms)"), 300))
  ];

  console.log("Iterating promises using for await...of:");
  for await (const item of promises) {
    console.log("- Received item:", item);
  }
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Sử dụng cú pháp vòng lặp for await...of (Async Iteration) để duyệt qua danh sách Promise.

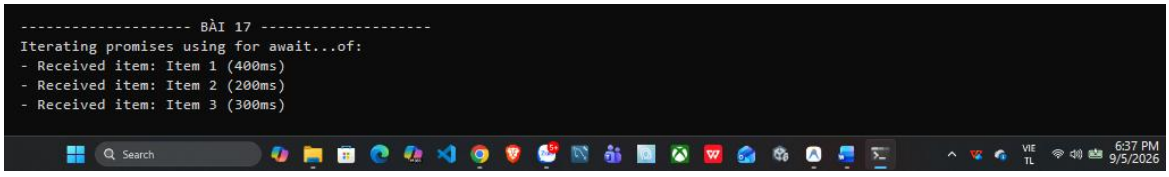
2. Phân tích chi tiết mã nguồn:

- Khởi tạo một mảng chứa 3 Promise có thời gian hoàn tất khác nhau (400ms, 200ms, 300ms).

- Cú pháp for await (const item of promises) lần lượt đợi từng Promise giải quyết xong theo đúng thứ tự phần tử trong mảng (Item 1 -> Item 2 -> Item 3).

3. Ứng dụng thực tế: Rất hữu ích khi xử lý dữ liệu truyền theo dòng (Streams), đọc file từng chunk lớn trên thiết bị, hoặc xử lý các trang dữ liệu (Pagination) tuần tự.

ẢNH CHỤP KẾT QUẢ



BÀI 18 – Async function fetchUser(id) giả lập gọi API

Yêu cầu đề bài: Write an async function fetchUser(id) that simulates an API call (resolves a user object after 1 second).

SOURCE CODE

```
// BÀI 18: Async function fetchUser(id) giả lập API (resolve user sau 1s)
export interface User {
  id: number;
  name: string;
  email: string;
  role: string;
}

export async function fetchUser(id: number): Promise<User> {
  await new Promise((resolve) => setTimeout(resolve, 1000));
  return {
    id,
    name: `Student_${id}`,
    email: `student${id}@iuh.edu.vn`,
    role: "Undergraduate"
  };
}

export async function bai18(): Promise<void> {
  console.log("Fetching user 101...");
  const user = await fetchUser(101);
  console.log("Fetched User details:", user);
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Xây dựng interface User và hàm fetchUser(id) mô phỏng lời gọi REST API lấy thông tin người dùng.

2. Phân tích chi tiết mã nguồn:

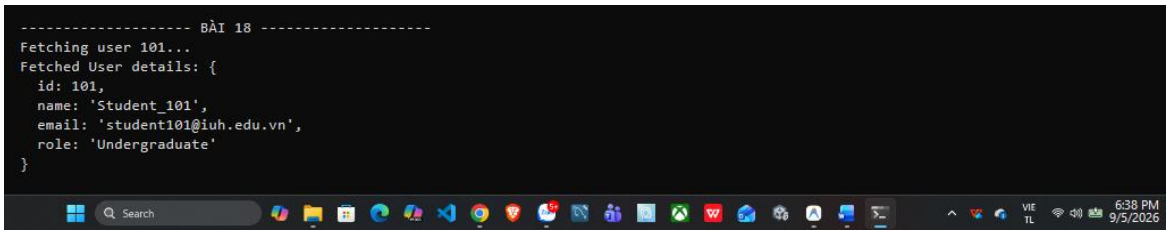
- Định nghĩa interface User rõ ràng với các trường: id, name, email, role.

- Hàm fetchUser(id) tạo độ trễ mạng 1000ms bằng setTimeout, sau đó trả về một đối tượng thỏa mãn interface User.

- Hàm bai18() gọi await fetchUser(101) và in ra đầy đủ thông tin đối tượng sinh viên nhận được.

3. Ý nghĩa thực tế: Chuẩn hóa kiểu dữ liệu truyền nhận giữa Mobile App và Backend Server, tận dụng tối đa thế mạnh kiểm soát kiểu tĩnh (Type Safety) của TypeScript.

ẢNH CHỤP KẾT QUẢ



BÀI 19 – Async function fetchUsers(ids) gọi fetchUser cho từng ID

Yêu cầu đề bài: Create an async function fetchUsers(ids: number[]) that calls fetchUser for each ID.

SOURCE CODE

```
// BÀI 19: Async function fetchUsers(ids) gọi fetchUser cho mỗi ID
import { fetchUser, User } from "./bai18";

export async function fetchUsers(ids: number[]): Promise<User[]> {
  const promises = ids.map((id) => fetchUser(id));
  return await Promise.all(promises);
}

export async function bai19(): Promise<void> {
  const ids = [1, 2, 3];
  console.log(`Fetching multiple users for IDs: [${ids.join(", ")}]...`);
  const users = await fetchUsers(ids);
  console.log("Fetched all users successfully:");
  users.forEach((u) => console.log(` - [${u.id}] ${u.name} (${u.email})`));
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Viết hàm fetchUsers nhận vào mảng ID và tải thông tin của nhiều người dùng song song.

2. Phân tích chi tiết mã nguồn:

- Hàm fetchUsers(ids: number[]) nhận mảng ID (ví dụ: [1, 2, 3]).

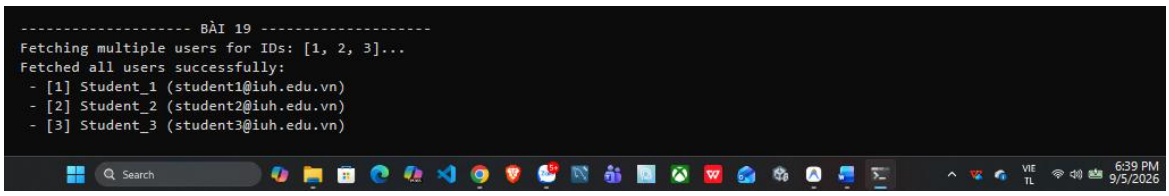
- Sử dụng ids.map(id => fetchUser(id)) để tạo ra một mảng các Promise<User>.

- Dùng await Promise.all(promises) để đồng loạt gửi các request. Dù lấy 3 user nhưng thời gian chỉ mất 1 giây thay vì 3 giây.

- Kết quả trả về là mảng User[] hoàn chỉnh, sau đó dùng forEach duyệt và hiển thị thông tin từng sinh viên.

3. Ứng dụng thực tế: Thường dùng trong màn hình danh sách bạn bè, danh sách chat, nơi ứng dụng cần tải thông tin chi tiết cho một loạt tài khoản cùng lúc.

ẢNH CHỤP KẾT QUẢ



BÀI 20 – Kỹ thuật Timeout: Quăng lỗi nếu API mất hơn 2 giây

Yêu cầu đề bài: Add a timeout: if the API call takes more than 2 seconds, throw an error.

SOURCE CODE

```
// BÀI 20: Thêm cơ chế timeout: quăng lỗi nếu API chạy quá 2 giây
export async function withTimeout<T>(promise: Promise<T>, timeoutMs: number = 2000):
Promise<T> {
  let timer: NodeJS.Timeout;
  const timeoutPromise = new Promise<never>((_, reject) => {
    timer = setTimeout(() => {
      reject(new Error(`Operation timed out after ${timeoutMs}ms`));
    }, timeoutMs);
  });

  return Promise.race([
    promise.then((val) => {
      clearTimeout(timer);
      return val;
    }),
    timeoutPromise
  ]);
}

export async function bai20(): Promise<void> {
  const slowApiCall = (): Promise<string> =>
    new Promise((resolve) => setTimeout(() => resolve("Slow data"), 3000));

  try {
    console.log("Executing slow API call with 2000ms timeout threshold...");
    await withTimeout(slowApiCall(), 2000);
  } catch (error: any) {
    console.log("[Timeout Handled]:", error.message);
  }
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Kỹ thuật thiết lập giới hạn thời gian chờ (Timeout Pattern) cho lời gọi API để tránh treo ứng dụng.

2. Phân tích chi tiết mã nguồn:

- Hàm withTimeout nhận vào một Promise tác vụ chính và khoảng thời gian timeoutMs (mặc định 2000ms).

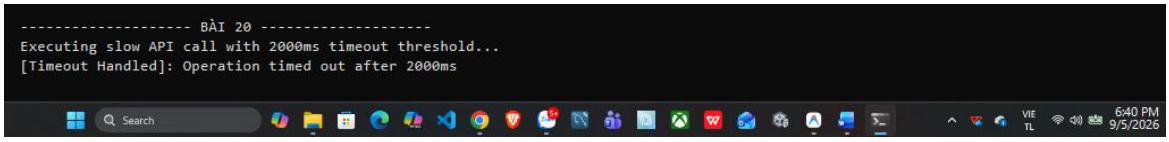
- Tạo một timeoutPromise sẽ reject lỗi sau 2000ms. Dùng Promise.race để cho 2 Promise này cạnh tranh nhau.

- Tác vụ slowApiCall cần tới 3000ms mới hoàn thành. Do đó, timeoutPromise sẽ về đích trước sau 2000ms và kích hoạt lỗi 'Operation timed out after 2000ms'.

- Ngoại lệ được khối try/catch bắt giữ và hiển thị thông báo xử lý lỗi rõ ràng.

3. Tầm quan trọng trong di động: Mạng di động (3G/4G/5G/Wifi) thường xuyên bị gián đoạn. Thiết lập timeout giúp ứng dụng kịp thời thông báo cho người dùng thay vì để màn hình đơ vô hạn.

ẢNH CHỤP KẾT QUẢ


---------- BÀI 20 ----------
Executing slow API call with 2000ms timeout threshold...
[Timeout Handled]: Operation timed out after 2000ms
Search
VIE
TL
6:40 PM
9/5/2026

BÀI 21 – Dùng fetch lấy dữ liệu từ Public API

Yêu cầu đề bài: Use fetch to get data from a public API (e.g., https://jsonplaceholder.typicode.com/todos/1).

SOURCE CODE

```
// BÀI 21: Dùng fetch lấy dữ liệu từ public API
export interface TodoItem {
  userId: number;
  id: number;
  title: string;
  completed: boolean;
}

export async function bai21(): Promise<TodoItem> {
  const url = "https://jsonplaceholder.typicode.com/todos/1";
  console.log("Fetching GET:", url);
  const response = await fetch(url);
  if (!response.ok) {
    throw new Error(`HTTP error! Status: ${response.status}`);
  }
  const data: TodoItem = await response.json();
  console.log("Fetched Todo Item:", data);
  return data;
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Sử dụng Fetch API tiêu chuẩn để gửi HTTP GET request đến Public REST API thật (JSONPlaceholder).

2. Phân tích chi tiết mã nguồn:

- Định nghĩa interface TodoItem gồm userId, id, title, completed.

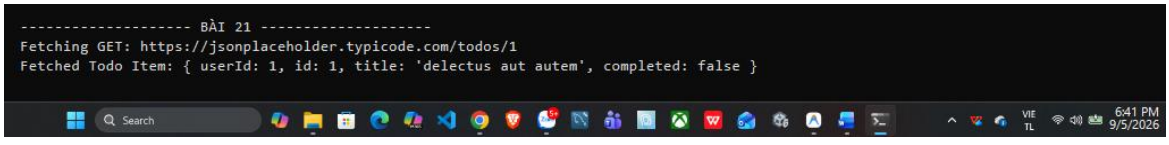
- Gửi request: const response = await fetch("https://jsonplaceholder.typicode.com/todos/1").

- Kiểm tra tính hợp lệ qua response.ok (status code 200..299). Nếu không hợp lệ thì quăng lỗi HTTP status.

- Chuyển đổi dữ liệu JSON: const data: TodoItem = await response.json().

3. Ứng dụng thực tế: Fetch API là chuẩn kết nối mạng cơ bản nhất trong JavaScript/TypeScript hiện đại, được hỗ trợ gốc trong Node.js 18+ cũng như React Native và các framework di động.

ẢNH CHỤP KẾT QUẢ



BÀI 22 – Gọi API nhiều lần và in kết quả

Yêu cầu đề bài: Call the API multiple times and log the results.

SOURCE CODE

```
// BÀI 22: Gọi API nhiều lần và in kết quả
import { TodoItem } from "./bai21";

export async function bai22(): Promise<void> {
  const ids = [1, 2, 3];
  console.log("Calling API for multiple todos...");
  const promises = ids.map(async (id) => {
    const res = await fetch(`https://jsonplaceholder.typicode.com/todos/${id}`);
    return res.json() as Promise<TodoItem>;
  });
  const results = await Promise.all(promises);
  results.forEach((todo) => {
    console.log(`- Todo #${todo.id}: "${todo.title}" | Completed: ${todo.completed}`);
  });
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Gửi nhiều request HTTP GET đến REST API cùng lúc và xử lý danh sách kết quả trả về.

2. Phân tích chi tiết mã nguồn:

- Tạo mảng các ID cần lấy [1, 2, 3].

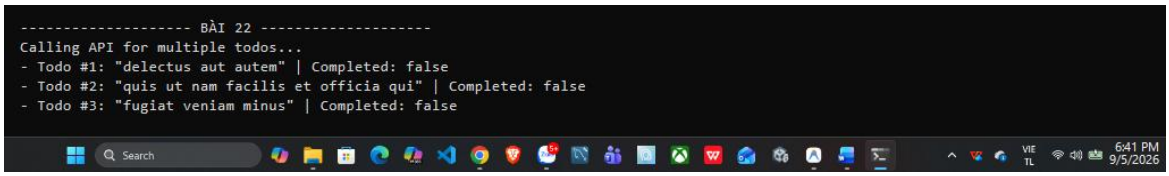
- Ánh xạ từng ID thành một hàm fetch bất đồng bộ gọi endpoint tương ứng /todos/{id}.

- Sử dụng Promise.all để gửi đồng loạt 3 HTTP request qua mạng Internet.

- Lặp qua mảng kết quả nhận được và log ra console thông tin ID, tiêu đề và trạng thái hoàn thành của từng todo.

3. Đánh giá hiệu năng: Giảm thiểu đáng kể hiện tượng nghẽn mạng và thời gian phản hồi so với việc gọi tuần tự từng request đơn lẻ.

ẢNH CHỤP KẾT QUẢ



BÀI 23 – Lấy danh sách todos và lọc các việc chưa hoàn thành

Yêu cầu đề bài: Write an async function that fetches a list of todos and filters out those that are not completed.

SOURCE CODE

```
// BÀI 23: Lấy danh sách todos và lọc ra các việc chưa hoàn thành
import { TodoItem } from "./bai21";

export async function bai23(): Promise<TodoItem[]> {
  const url = "https://jsonplaceholder.typicode.com/todos?_limit=8";
  console.log("Fetching todos from:", url);
  const response = await fetch(url);
  const todos: TodoItem[] = await response.json();

  const uncompleted = todos.filter((todo) => !todo.completed);
  console.log(`Found ${uncompleted.length} incompleted todos out of ${todos.length}:`);
  uncompleted.forEach((t) => console.log(`  * [#${t.id}] ${t.title}`));
  return uncompleted;
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Tải danh sách công việc từ API và áp dụng phương thức filter để trích xuất các công việc chưa hoàn thành.

2. Phân tích chi tiết mã nguồn:

- Gửi GET request kèm query parameter: /todos?_limit=8 để lấy về 8 công việc đầu tiên.

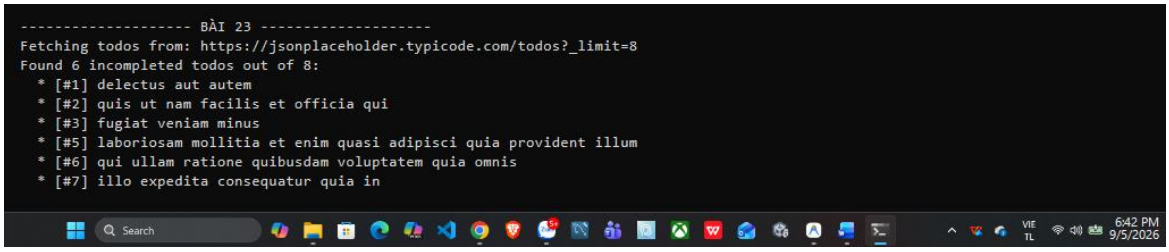
- Dùng await response.json() phân giải mảng dữ liệu TodoItem[].

- Áp dụng phương thức mảng todos.filter(todo => !todo.completed) để lọc các mục có trường completed là false.

- In ra số lượng công việc chưa hoàn thành (6/8 mục) và hiển thị chi tiết từng công việc.

3. Ý nghĩa thực tế: Tái hiện chức năng quản lý tác vụ (To-Do List) rất quen thuộc trên các ứng dụng di động, kết hợp nhuần nhuyễn giữa xử lý mạng và xử lý mảng dữ liệu.

ẢNH CHỤP KẾT QUẢ



BÀI 24 – Async function postData() gửi HTTP POST request

Yêu cầu đề bài: Write an async function postData() that sends a POST request to a test API.

SOURCE CODE

```
// BÀI 24: Async function postData() gửi POST request đến test API
export interface PostData {
  id?: number;
  title: string;
  body: string;
  userId: number;
}

export async function postData(): Promise<PostData> {
  const payload: PostData = {
    title: "Bai tap Thuc hanh TBDD - IUH",
    body: "Thuc hanh gui du lieu POST len REST API voi TypeScript",
    userId: 23695731
  };

  const response = await fetch("https://jsonplaceholder.typicode.com/posts", {
    method: "POST",
    headers: {
      "Content-Type": "application/json; charset=UTF-8"
    },
    body: JSON.stringify(payload)
  });

  const createdPost: PostData = await response.json();
  return createdPost;
}

export async function bai24(): Promise<void> {
  console.log("Sending POST request to create new post...");
  const result = await postData();
  console.log("Server response after creation:", result);
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Gửi HTTP POST request với thân (Body) dữ liệu dạng JSON lên máy chủ REST API.

2. Phân tích chi tiết mã nguồn:

- Định nghĩa interface PostData (title, body, userId, id).

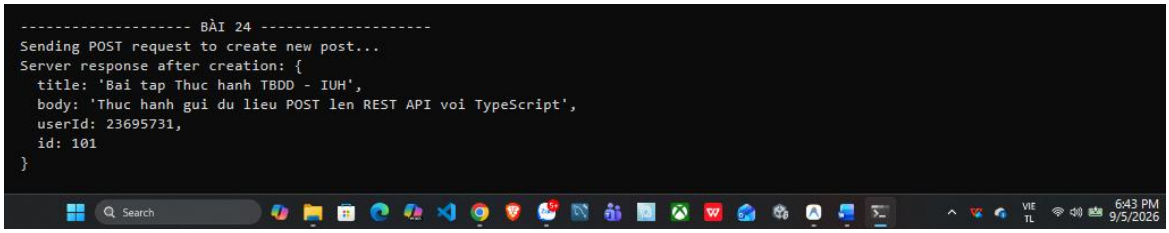
- Tạo đối tượng payload với dữ liệu thực hành của sinh viên IUH.

- Cấu hình hàm fetch với tùy chọn method: "POST", headers gán Content-Type là application/json và body chuyển thành chuỗi JSON qua JSON.stringify(payload).

- Server JSONPlaceholder nhận dữ liệu, sinh ID mới (101) và phản hồi lại đối tượng vừa tạo.

3. Ứng dụng thực tế: Phương thức POST là nền tảng để gửi biểu mẫu đăng ký, tạo bài đăng mới, bình luận hoặc tải thông tin cá nhân lên hệ sinh thái ứng dụng.

ẢNH CHỤP KẾT QUẢ


-------------------- BÀI 24 --------------------
Sending POST request to create new post...
Server response after creation: {
title: 'Bai tap Thuc hanh TBDD - IUH',
body: 'Thuc hanh gui du lieu POST len REST API voi TypeScript',
userId: 23695731,
id: 101
}
Search
VIE
TL
6:43 PM
9/5/2026

BÀI 25 – Hàm downloadFile mô phỏng tải file trong 3 giây

Yêu cầu đề bài: Create a function downloadFile that simulates downloading a file in 3 seconds and logs when done.

SOURCE CODE

```
// BÀI 25: Hàm downloadFile mô phỏng tải file trong 3 giây
export async function downloadFile(filename: string): Promise<string> {
  console.log(`[Start] Downloading file: "${filename}"...`);
  await new Promise((resolve) => setTimeout(resolve, 3000));
  const doneMsg = `[Done] Successfully downloaded "${filename}" (3 seconds).`;
  console.log(doneMsg);
  return doneMsg;
}

export async function bai25(): Promise<void> {
  await downloadFile("Android_ReactNative_Guide.pdf");
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Viết hàm downloadFile mô phỏng tiến trình tải tập tin trong 3 giây và thông báo khi hoàn tất.

2. Phân tích chi tiết mã nguồn:

- Hàm nhận tham số filename đại diện cho tên file cần tải (ví dụ: 'Android_ReactNative_Guide.pdf').

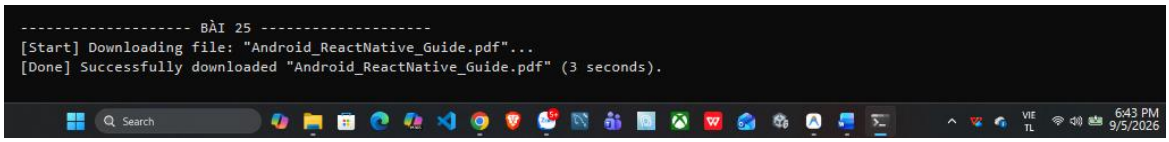
- In ra console thông báo bắt đầu tải tệp.

- Dùng await new Promise(resolve => setTimeout(resolve, 3000)) để giữ luồng chờ chính xác 3 giây.

- Khi hết thời gian, in thông báo thành công và trả về chuỗi xác nhận.

3. Ứng dụng thực tế: Mô phỏng quá trình tải tài nguyên media (tải ảnh, file âm thanh, cập nhật tệp tin OTA) trong ứng dụng di động, giúp lập trình viên kiểm soát luồng giao diện người dùng.

ẢNH CHỤP KẾT QUẢ



BÀI 26 – Dùng async/await với setTimeout mô phỏng chờ 5 giây

Yêu cầu đề bài: Use async/await with setTimeout to simulate a 5-second wait.

SOURCE CODE

```
// BÀI 26: Dùng async/await với setTimeout mô phỏng chờ 5 giây
export async function sleep(ms: number): Promise<void> {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

export async function bai26(): Promise<void> {
  console.log("Starting 5-second delay...");
  const t0 = Date.now();
  await sleep(5000);
  const elapsed = ((Date.now() - t0) / 1000).toFixed(2);
  console.log(`Finished waiting. Total time elapsed: ${elapsed}s`);
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Sử dụng async/await kết hợp setTimeout để xây dựng hàm tạm dừng thực thi (sleep) 5 giây.

2. Phân tích chi tiết mã nguồn:

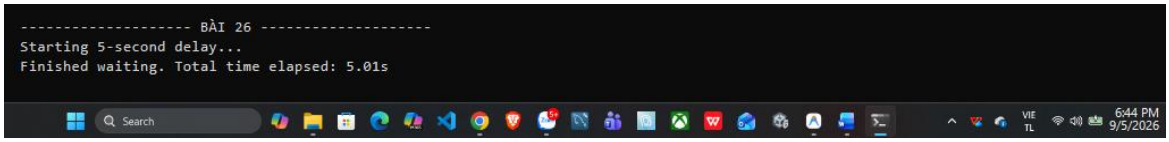
- Xây dựng hàm sleep(ms: number) trả về một Promise tự động resolve sau ms mili-giây.

- Trong hàm bai26, ghi lại mốc thời gian bắt đầu bằng Date.now().

- Lệnh await sleep(5000) dừng hàm đúng 5 giây. Sau đó đo thời gian đã trôi qua và in ra console (~5.01s).

3. Đặc điểm cốt lõi: Lệnh sleep này hoàn toàn không chặn (non-blocking) các tiến trình khác trên hệ thống, khác hẳn với các vòng lặp while bận (busy-wait) gây quá tải CPU.

ẢNH CHỤP KẾT QUẢ



BÀI 27 – Hàm fetchWithRetry(url, retries) tự động thử lại khi lỗi

Yêu cầu đề bài: Write a function fetchWithRetry(url, retries) that retries up to retries times if the API call fails.

SOURCE CODE

```
// BÀI 27: Viết hàm fetchWithRetry(url, retries) tự động thử lại nếu lỗi
export async function fetchWithRetry<T>(url: string, retries: number): Promise<T> {
  for (let attempt = 1; attempt <= retries; attempt++) {
    try {
      console.log(`Attempt ${attempt}/${retries}: Fetching ${url}...`);
      const response = await fetch(url);
      if (!response.ok) {
        throw new Error(`HTTP ${response.status}: ${response.statusText}`);
      }
      return (await response.json()) as T;
    } catch (error: any) {
      console.log(`  -> Attempt ${attempt} failed: ${error.message}`);
      if (attempt === retries) {
        throw new Error(`Failed to fetch after ${retries} attempts: ${error.message}`);
      }
      await new Promise((r) => setTimeout(r, 800)); // Chờ 800ms trước khi thử lại
    }
  }
  throw new Error("Unable to connect.");
}

export async function bai27(): Promise<void> {
  const invalidUrl = "https://jsonplaceholder.typicode.com/invalid-endpoint-test-404";
  try {
    await fetchWithRetry(invalidUrl, 3);
  } catch (err: any) {
    console.log("[Final Catch]:", err.message);
  }
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Hiện thực hóa mẫu thiết kế Retry Pattern để tự động thử lại khi gọi API thất bại.

2. Phân tích chi tiết mã nguồn:

- Hàm fetchWithRetry<T>(url, retries) sử dụng vòng lặp for chạy từ 1 đến số lần retries cho phép.

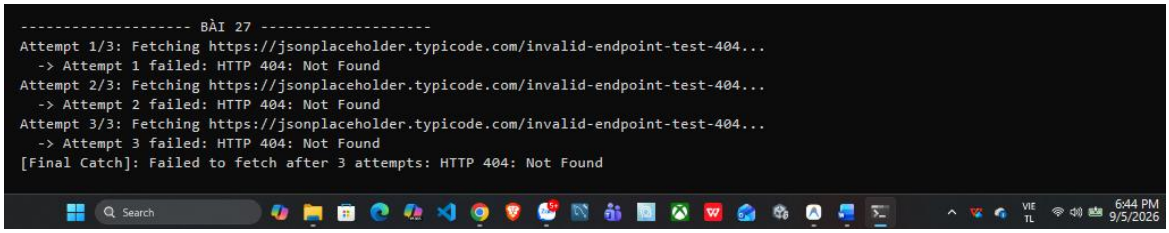
- Trong mỗi lần thử, khối try gửi request tới url. Nếu response.ok thì trả về dữ liệu ngay lập tức.

- Nếu phát sinh lỗi (mạng đứt, HTTP 404), khối catch ghi nhận và kiểm tra nếu chưa đến lần thử cuối thì chờ 800ms rồi tiếp tục vòng lặp mới.

- Nếu đã thử đủ số lần quy định mà vẫn thất bại thì mới quăng ra ngoại lệ cuối cùng.

3. Ứng dụng thực tế: Cực kỳ quan trọng trong lập trình di động để đảm bảo độ tin cậy của ứng dụng khi thiết bị đi qua vùng sóng yếu hoặc chuyển vùng mạng.

ẢNH CHỤP KẾT QUẢ


-------------------- BÀI 27 --------------------
Attempt 1/3: Fetching https://jsonplaceholder.typicode.com/invalid-endpoint-test-404...
-> Attempt 1 failed: HTTP 404: Not Found
Attempt 2/3: Fetching https://jsonplaceholder.typicode.com/invalid-endpoint-test-404...
-> Attempt 2 failed: HTTP 404: Not Found
Attempt 3/3: Fetching https://jsonplaceholder.typicode.com/invalid-endpoint-test-404...
-> Attempt 3 failed: HTTP 404: Not Found
[Final Catch]: Failed to fetch after 3 attempts: HTTP 404: Not Found
Search
VIE
TL
6:44 PM
9/5/2026

BÀI 28 – Hàm batchProcess() xử lý 5 tác vụ cùng lúc với Promise.all

Yêu cầu đề bài: Write an async function batchProcess() that processes 5 async tasks at once (use Promise.all).

SOURCE CODE

```
// BÀI 28: Async function batchProcess() xử lý 5 tác vụ đồng thời
import { simulateTask } from "./bai05";

export async function batchProcess(): Promise<string[]> {
  const taskIds = [1, 2, 3, 4, 5];
  const tasks = taskIds.map((id) =>
    simulateTask(400 + id * 100).then(() => `Batch task #${id} done`)
  );
  return await Promise.all(tasks);
}

export async function bai28(): Promise<void> {
  console.log("Processing batch of 5 asynchronous tasks...");
  const results = await batchProcess();
  console.log("Batch completed successfully:", results);
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Xây dựng hàm batchProcess() để xử lý đồng thời một lô (batch) gồm 5 tác vụ bất đồng bộ.

2. Phân tích chi tiết mã nguồn:

- Tạo mảng gồm 5 mã công việc [1, 2, 3, 4, 5].

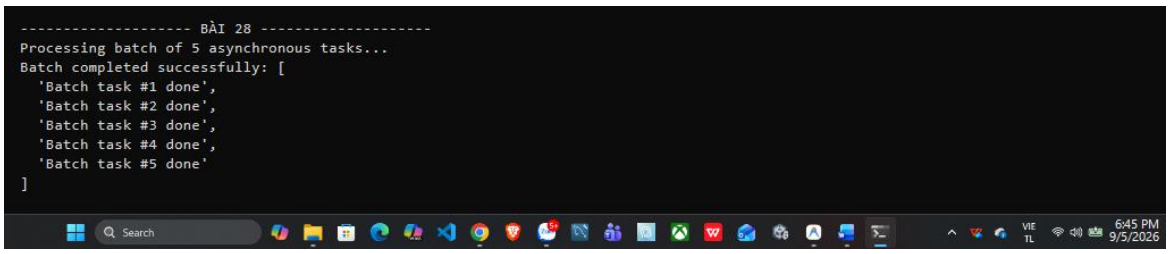
- Dùng array.map chuyển mỗi phần tử thành một Promise gọi simulateTask với thời gian trễ tăng dần.

- await Promise.all(tasks) thực thi đồng thời cả 5 tác vụ trong lô và chờ toàn bộ hoàn thành.

- Kết quả trả về là mảng gồm 5 thông báo hoàn thành của từng tác vụ.

3. Ý nghĩa thực tế: Giúp gộp các tác vụ nhỏ lẻ (như đồng bộ nhiều ảnh lên cloud, gửi hàng loạt thông báo) vào các đợt xử lý tập trung để tiết kiệm băng thông và tối ưu CPU thiết bị di động.

ẢNH CHỤP KẾT QUẢ



BÀI 29 – Hàm queueProcess() xử lý hàng đợi tác vụ tuần tự

Yêu cầu đề bài: Write an async function queueProcess() that processes tasks sequentially in a queue.

SOURCE CODE

```
// BÀI 29: Async function queueProcess() xử lý tác vụ tuần tự theo hàng đợi
export type TaskGenerator<T> = () => Promise<T>;

export async function queueProcess<T>(tasks: TaskGenerator<T>[]): Promise<T[]> {
  const results: T[] = [];
  for (let i = 0; i < tasks.length; i++) {
    console.log(`[Queue] Processing job ${i + 1}/${tasks.length}...`);
    const res = await tasks[i]();
    results.push(res);
    console.log(`[Queue] Job ${i + 1} completed: ${res}`);
  }
  return results;
}

export async function bai29(): Promise<void> {
  const tasks: TaskGenerator<string>[] = [
    () => new Promise((resolve) => setTimeout(() => resolve("Task Alpha initialized"),
300)),
    () => new Promise((resolve) => setTimeout(() => resolve("Task Beta synced"), 200)),
    () => new Promise((resolve) => setTimeout(() => resolve("Task Gamma finalized"), 250))
  ];

  console.log("Starting queue execution...");
  const results = await queueProcess(tasks);
  console.log("Queue execution finished with results:", results);
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Hiện thực hóa cơ chế hàng đợi (Queue Processing) để xử lý các tác vụ bất đồng bộ theo đúng thứ tự First-In-First-Out (FIFO).

2. Phân tích chi tiết mã nguồn:

- Định nghĩa kiểu TaskGenerator<T> = () => Promise<T> đại diện cho một hàm sinh ra tác vụ bất đồng bộ.

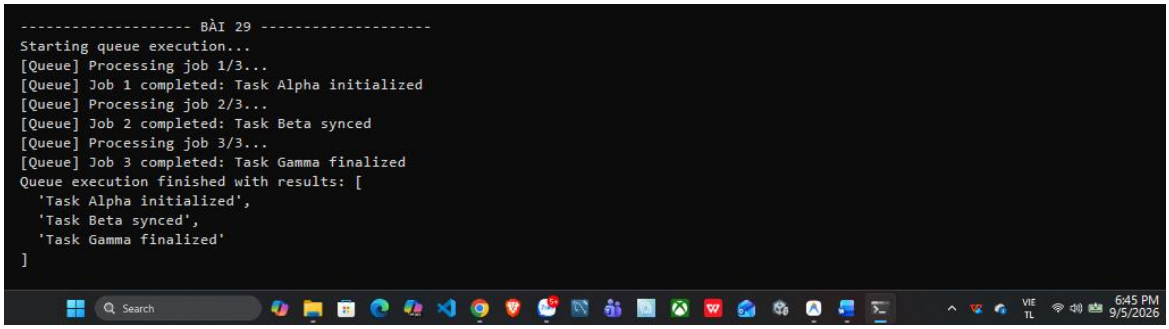
- Hàm queueProcess nhận vào một mảng các TaskGenerator, sử dụng vòng lặp for tuần tự kết hợp await tasks[i]().

- Tác vụ i + 1 chỉ bắt đầu thực thi khi tác vụ i đã hoàn thành trọn vẹn.

- Kết quả từng bước được lưu vào mảng results và trả về ở cuối hàm.

3. Ứng dụng thực tế: Rất cần thiết cho các tác vụ mang tính giao dịch (Database Transactions), ghi log tuần tự, hoặc xử lý hàng đợi thanh toán trên điện thoại nhằm ngăn chặn hiện tượng Race Condition.

ẢNH CHỤP KẾT QUẢ


-------------------- BÀI 29 --------------------
Starting queue execution...
[Queue] Processing job 1/3...
[Queue] Job 1 completed: Task Alpha initialized
[Queue] Processing job 2/3...
[Queue] Job 2 completed: Task Beta synced
[Queue] Processing job 3/3...
[Queue] Job 3 completed: Task Gamma finalized
Queue execution finished with results: [
'Task Alpha initialized',
'Task Beta synced',
'Task Gamma finalized'
]
Search
VIE
TL
6:45 PM
9/5/2026

BÀI 30 – Dùng Promise.allSettled() kiểm tra trạng thái nhiều API

Yêu cầu đề bài: Use async/await + Promise.allSettled() to handle multiple API calls and display their success/failure status.

SOURCE CODE

```
// BÀI 30: Dùng async/await + Promise.allSettled() xử lý nhiều API call
export async function bai30(): Promise<void> {
  const urls = [
    "https://jsonplaceholder.typicode.com/todos/1",
    "https://jsonplaceholder.typicode.com/non-existing-404-endpoint",
    "https://jsonplaceholder.typicode.com/todos/2"
  ];

  console.log("Checking API endpoints with Promise.allSettled()...");
  const promises = urls.map(async (url) => {
    const res = await fetch(url);
    if (!res.ok) throw new Error(`HTTP ${res.status} (${res.statusText})`);
    return res.json();
  });

  const results = await Promise.allSettled(promises);
  results.forEach((result, idx) => {
    if (result.status === "fulfilled") {
      console.log(`Endpoint #${idx + 1} [${urls[idx]}]: SUCCESS ->`, result.value);
    } else {
      console.log(`Endpoint #${idx + 1} [${urls[idx]}]: FAILED ->
${result.reason.message}`);
    }
  });
}
```

GIẢI THÍCH

1. Khái niệm và mục tiêu: Sử dụng Promise.allSettled() để gửi nhiều API request và kiểm soát chi tiết trạng thái thành công/thất bại của từng endpoint.

2. Phân tích chi tiết mã nguồn:

- Chuẩn bị 3 URL trong đó có 2 endpoint hợp lệ và 1 endpoint lỗi cố tình (404 Not Found).

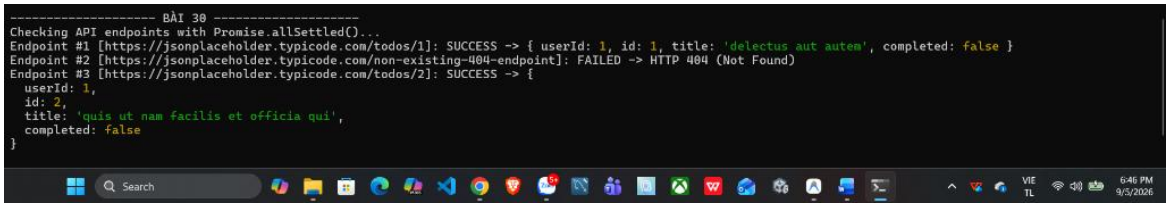
- Chuyển danh sách URL thành mảng Promise gọi fetch và truyền cho Promise.allSettled(promises).

- Khác với Promise.all (sẽ dừng ngay nếu có lỗi), Promise.allSettled luôn chờ cả 3 request hoàn thành và trả về mảng kết quả có thuộc tính status là 'fulfilled' (kèm value) hoặc 'rejected' (kèm reason).

- Dùng forEach duyệt qua để in rõ endpoint nào THÀNH CÔNG và endpoint nào THẤT BẠI cùng nguyên nhân chi tiết.

3. Tầm quan trọng: Giúp lập trình viên di động xây dựng các bảng điều khiển trạng thái hệ thống (Dashboard/Healthcheck), cho phép một phần giao diện vẫn hiển thị dữ liệu bình thường dù một dịch vụ con gặp sự cố.

ẢNH CHỤP KẾT QUẢ


---------- BÀI 30 ----------
Checking API endpoints with Promise.allSettled()...
Endpoint #1 [https://jsonplaceholder.typicode.com/todos/1]: SUCCESS -> { userId: 1, id: 1, title: 'delectus aut autem', completed: false }
Endpoint #2 [https://jsonplaceholder.typicode.com/non-existing-404-endpoint]: FAILED -> HTTP 404 (Not Found)
Endpoint #3 [https://jsonplaceholder.typicode.com/todos/2]: SUCCESS -> {
userId: 1,
id: 2,
title: 'quis ut nam facilis et officia qui',
completed: false
}
Search
VIE
TL
6:46 PM
9/5/2026